



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: Summer, Year: 2026, B.Sc. in CSE (Day)

Sorting Algorithm Visualizer

Project Report

Course Title: Algorithms Lab
Course Code: CSE 208 Section: 251-D4

Student Details:

Name	ID
Jonayet Bogdady	251002011

Submission Date: 19/08/2026
Course Teacher's Name: Ms. Jarin Tasnim Tonvi

[For teachers to use only: **Don't write anything inside this box.**]

Project Report Status	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	3
1.1	Overview	3
1.2	Motivation	3
1.3	Problem Definition	4
1.3.1	Problem Statement	4
1.3.2	Complex Engineering Problem	4
1.4	Design Goals/Objectives	5
1.5	Application	5
2	Design, Development, and Implementation of the Project	6
2.1	Introduction	6
2.2	Project Details	6
2.2.1	System Architecture	6
2.2.2	User Interface and Tools	7
2.3	Implementation	7
2.3.1	Library and Global Variable Portion	7
2.3.2	Swap Portion	8
2.3.3	Visualize code portion	8
2.3.4	Bubble Sort Portion	9
2.3.5	Selection Sort Portion	9
2.3.6	Insertion Sort Portion	10
2.3.7	Quick Sort Portion	10
2.3.8	Merge Sort Portion	11
2.3.9	Input Array Portion	12
2.3.10	Random Array Portion	12
2.3.11	Time Complexity Portion	13
2.3.12	Main Code Portion	14

3	Performance Evaluation	16
3.1	Simulation Environment/Simulation Procedure	16
3.2	Results Analysis/Testing	16
3.3	Project Simulation	18
3.3.1	Manual Input Main Menu	18
3.3.2	Bubble Sort	18
3.3.3	Selection Sort	19
3.3.4	Insertion Sort	19
3.3.5	Merge Sort	20
3.3.6	Qucik Sort	20
3.3.7	Random Array Main Menu	21
3.3.8	End Program Portion	21
3.4	Results Overall Discussion	22
3.4.1	Complex Engineering Problem Discussion	22
4	Conclusion	23
4.1	Discussion	23
4.2	Limitations	23
4.3	Scope of Future Work	24
4.4	Conclusion	24

Chapter 1

Introduction

1.1 Overview

Sorting Algorithm Visualizer is a software project developed using the C programming language. The main purpose of this project is to show how different sorting algorithms work step by step. Many students find sorting algorithms difficult because they cannot see what happens during the sorting process. This project solves that problem by showing the movement of data while sorting.

The program will allow users to choose a sorting algorithm from a menu. After selecting an algorithm, the program will display each step of the sorting process. The user can compare different algorithms and understand their working process easily. The project includes the following sorting algorithms:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

1.2 Motivation

Sorting is one of the most important topics in computer science. Students usually learn sorting by reading code, but it is hard to understand how the algorithms actually work.

The motivation of this project is to make learning easier by showing the sorting process visually. When users can watch every comparison and swap, they understand the algorithm better. This project also helps students compare the speed and behavior of different sorting methods.

1.3 Problem Definition

1.3.1 Problem Statement

Many students have difficulty understanding sorting algorithms because they only see the program code or final output. They cannot easily understand how the data changes during sorting.

This project creates a visual learning tool that displays each step of the sorting process. It helps users learn, compare, and understand different sorting algorithms in a simple way. The project needs to solve several technical problems:

- Display the sorting process step by step.
- Update the screen after every comparison and swap.
- Handle different sorting algorithms in one program.
- Keep the animation smooth and easy to understand.
- Make the program simple and user-friendly.

1.3.2 Complex Engineering Problem

Table 1.1: Summary of the attributes touched by the mentioned projects

Name of the P Attributes	Explain how to address
P1: Depth of knowledge required	Knowledge of C programming, arrays, loops, functions, and sorting algorithms is required.
P2: Range of conflicting requirements	The program should be simple to use while also showing clear visualization
P3: Depth of analysis required	Compare different sorting algorithms based on their working process and efficiency.
P4: Familiarity of issues	Understand common problems like swapping elements, comparisons, and execution time.
P5: Extent of applicable codes	Use standard C programming and basic console functions.
P6: Extent of stakeholder involvement and conflicting requirements	Students and teachers need an easy learning tool with clear visualization.
P7: Interdependence	The visualization depends on correct implementation of each sorting algorithm and screen updates.

1.4 Design Goals/Objectives

The main objectives of this project are:

- To develop a sorting algorithm visualizer using C.
- To visualize sorting algorithms step by step.
- To help students understand sorting algorithms easily.
- To compare different sorting algorithms.
- To create a simple and user-friendly interface.
- To improve programming and algorithm learning.

1.5 Application

This project can be used in many areas, such as:

- Teaching sorting algorithms in programming classes.
- Learning data structures and algorithms.
- Laboratory demonstrations.
- Self-learning for beginner programmers.
- Comparing different sorting algorithms for educational purposes.

Chapter 2

Design, Development, and Implementation of the Project

2.1 Introduction

This chapter explains the design and implementation of the Sorting Algorithm Visualizer project. The visualizer was developed using the C programming language. It provides a menu-driven interface where users can input array elements, choose a sorting algorithm, and watch the sorting process step by step. The program displays the array state, comparisons, swaps, and execution time after every operation.

2.2 Project Details

2.2.1 System Architecture

The Sorting Algorithm Visualizer follows a simple modular design. Each module performs a specific task, making the program easier to understand and maintain.

The main modules of the system are:

1. Array Input Module: Handles manual entry or random generation of array elements
2. Sorting Algorithm Module: Implements the five sorting algorithms
3. Visualization Module: Displays the bar chart representation of the array
4. Statistics Module: Tracks comparisons, swaps, and execution time
5. Complexity Display Module: Shows theoretical complexity for each algorithm

2.2.2 User Interface and Tools

The project uses a text-based console interface. The user can perform the following tasks:

- Enter array elements manually
- Generate random array elements
- Select a sorting algorithm from the menu
- Watch the sorting process step by step
- See the number of comparisons and swaps
- View the execution time
- See the theoretical complexity of the selected algorithm

Development Tools:

- Compiler: GCC (MinGW)
- Development Environment: Visual Studio Code / Code::Blocks
- Operating System: Windows 10/11
- Libraries Used: stdio.h, stdlib.h, time.h, windows.h

2.3 Implementation

2.3.1 Library and Global Variable Portion

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #include <windows.h>
5
6  #define MAX 50
7
8  int arr[MAX];
9  int n;
10
11 long long comparisons = 0;
12 long long swaps = 0;
```

Figure 2.1: Library and Global Variable

2.3.2 Swap Portion

```
57 void swap(int *a, int *b)
58 {
59     int temp = *a;
60     *a = *b;
61     *b = temp;
62
63     swaps++;
64 }
```

Figure 2.2: Swap Function

2.3.3 Visualize code portion

```
14 void delay()
15 {
16     Sleep(0000);
17 }
18
19 void printArray()
20 {
21     int i;
22
23     printf("\nArray: ");
24
25     for (i = 0; i < n; i++)
26     {
27         printf("%d ", arr[i]);
28     }
29
30     printf("\n");
31 }
32
33 void visualize()
34 {
35     int i, j;
36
37     system("cls");
38
39     printf("\033[1;36m===== Sorting Algorithm Visualizer =====\033[0m\n\n");
40
41     for (i = 0; i < n; i++)
42     {
43         printf("%2d | ", arr[i]);
44
45         for (j = 0; j < arr[i]; j++)
46             printf("\033[0;30;44m-\033[0m");
47
48         printf("\n");
49     }
50
51     printf("\nComparisons : %lld", comparisons);
52     printf("\nSwaps      : %lld\n", swaps);
53
54     delay();
55 }
--
```

Figure 2.3: Visualization of Sorting Algorithm Function

2.3.4 Bubble Sort Portion

```
64 void bubbleSort()
65 {
66     for (int i = 0; i < n - 1; i++)
67     {
68         int swapped = 0;
69
70         for (int j = 0; j < n - i - 1; j++)
71         {
72             comparisons++;
73
74             if (arr[j] > arr[j + 1])
75             {
76                 swap(&arr[j], &arr[j + 1]);
77                 swapped = 1;
78
79                 visualize();
80             }
81         }
82
83         if (swapped == 0)
84             break;
85     }
86 }
```

Figure 2.4: Bubble Sort Function

2.3.5 Selection Sort Portion

```
88 void selectionSort()
89 {
90     int minIndex;
91
92     for (int i = 0; i < n - 1; i++)
93     {
94         minIndex = i;
95
96         for (int j = i + 1; j < n; j++)
97         {
98             comparisons++;
99
100             if (arr[j] < arr[minIndex])
101             {
102                 minIndex = j;
103             }
104         }
105
106         if (minIndex != i)
107         {
108             swap(&arr[i], &arr[minIndex]);
109             visualize();
110         }
111     }
112 }
```

Figure 2.5: Selection Sort Function

2.3.6 Insertion Sort Portion

```
114 void insertionSort()
115 {
116     int key;
117     for (int i = 1; i < n; i++)
118     {
119         key = arr[i];
120         int j = i - 1;
121         while (j >= 0)
122         {
123             comparisons++;
124             if (arr[j] > key)
125             {
126                 arr[j + 1] = arr[j];
127                 swaps++;
128                 visualize();
129                 j--;
130             }
131             else
132             {
133                 break;
134             }
135         }
136         arr[j + 1] = key;
137         swaps++;
138         visualize();
139     }
140 }
141
142
143
144
145
```

Figure 2.6: Insertion Sort Function

2.3.7 Quick Sort Portion

```
221 int partition(int low, int high)
222 {
223     int pivot = arr[high];
224     int i = low - 1;
225     int j;
226     for (j = low; j < high; j++)
227     {
228         comparisons++;
229         if (arr[j] < pivot)
230         {
231             i++;
232             swap(&arr[i], &arr[j]);
233             visualize();
234         }
235     }
236     swap(&arr[i + 1], &arr[high]);
237     visualize();
238     return i + 1;
239 }
240
241 void quickSort(int low, int high)
242 {
243     if (low < high)
244     {
245         int pi = partition(low, high);
246         quickSort(low, pi - 1);
247         quickSort(pi + 1, high);
248     }
249 }
250
251
252
253
254
---
```

Figure 2.7: Quick Sort Function

2.3.8 Merge Sort Portion

```
151 void merge(int left, int mid, int right)
152 {
153     int i, j, k;
154
155     int n1 = mid - left + 1;
156     int n2 = right - mid;
157
158     int L[MAX], R[MAX];
159
160     for (i = 0; i < n1; i++)
161         L[i] = arr[left + i];
162
163     for (j = 0; j < n2; j++)
164         R[j] = arr[mid + 1 + j];
165
166     i = 0;
167     j = 0;
168     k = left;
169
170     while (i < n1 && j < n2)
171     {
172         comparisons++;
173
174         if (L[i] <= R[j])
175         {
176             arr[k] = L[i];
177             i++;
178         }
179         else
180         {
181             arr[k] = R[j];
182             j++;
183         }
184
185         k++;
186         visualize();
187     }
188
189     while (i < n1)
190     {
191         arr[k] = L[i];
192         i++;
193         k++;
194
195         visualize();
196     }
197
198     while (j < n2)
199     {
200         arr[k] = R[j];
201         j++;
202         k++;
203
204         visualize();
205     }
206 }
207
208 void mergeSort(int left, int right)
209 {
210     if (left < right)
211     {
212         int mid = left + (right - left) / 2;
213
214         mergeSort(left, mid);
215         mergeSort(mid + 1, right);
216
217         merge(left, mid, right);
218     }
219 }
```

Figure 2.8: Merge Sort Function

2.3.9 Input Array Portion

```
256 void inputArray()
257 {
258     int i;
259
260     printf("\nEnter number of elements (1-%d): ", MAX);
261     scanf("%d", &n);
262
263     if (n < 1 || n > MAX)
264     {
265         printf("Invalid Size!\n");
266         exit(0);
267     }
268
269     printf("\nEnter Elements:\n");
270
271     for (i = 0; i < n; i++)
272     {
273         scanf("%d", &arr[i]);
274     }
```

Figure 2.9: Input Array Function

2.3.10 Random Array Portion

```
275 void randomArray()
276 {
277     int i;
278
279     printf("\nEnter number of elements (1-%d): ", MAX);
280     scanf("%d", &n);
281
282     if (n < 1 || n > MAX)
283     {
284         printf("Invalid Size!\n");
285         exit(0);
286     }
287
288     srand(time(NULL));
289
290     for (i = 0; i < n; i++)
291     {
292         arr[i] = rand() % 20 + 1;
293     }
294     printf("\nRandom Array Generated.\n");
```

Figure 2.10: Create Random Array Function

2.3.11 Time Complexity Portion

```
302 void showComplexity(int choice)
303 {
304     printf("\n\033[1;31m===== \033[0m\n");
305     switch (choice)
306     {
307     case 1:
308         printf("\033[1;31mBubble Sort\033[0m\n");
309         printf("\033[1;30mBest Case      : O(n)\033[0m\n");
310         printf("\033[1;30mAverage Case : O(n^2)\033[0m\n");
311         printf("\033[1;30mWorst Case   : O(n^2)\033[0m\n");
312         printf("\033[1;30mSpace        : O(1)\033[0m\n");
313         break;
314
315     case 2:
316         printf("\033[1;32mSelection Sort\033[0m\n");
317         printf("\033[1;30mBest Case      : O(n^2)\033[0m\n");
318         printf("\033[1;30mAverage Case : O(n^2)\033[0m\n");
319         printf("\033[1;30mWorst Case   : O(n^2)\033[0m\n");
320         printf("\033[1;30mSpace        : O(1)\033[0m\n");
321         break;
322
323     case 3:
324         printf("\033[1;34mInsertion Sort\033[0m\n");
325         printf("\033[1;30mBest Case      : O(n)\033[0m\n");
326         printf("\033[1;30mAverage Case : O(n^2)\033[0m\n");
327         printf("\033[1;30mWorst Case   : O(n^2)\033[0m\n");
328         printf("\033[1;30mSpace        : O(1)\033[0m\n");
329         break;
330
331     case 4:
332         printf("\033[1;35mMerge Sort\033[0m\n");
333         printf("\033[1;30mBest Case      : O(n log n)\033[0m\n");
334         printf("\033[1;30mAverage Case : O(n log n)\033[0m\n");
335         printf("\033[1;30mWorst Case   : O(n log n)\033[0m\n");
336         printf("\033[1;30mSpace        : O(n)\033[0m\n");
337         break;
338
339     case 5:
340         printf("\033[1;36mQuick Sort\033[0m\n");
341         printf("\033[1;30mBest Case      : O(n log n)\033[0m\n");
342         printf("\033[1;30mAverage Case : O(n log n)\033[0m\n");
343         printf("\033[1;30mWorst Case   : O(n^2)\033[0m\n");
344         printf("\033[1;30mSpace        : O(log n)\033[0m\n");
345         break;
346     }
347     printf("\033[1;31m===== \033[0m\n");
348 }
```

Figure 2.11: Time Complexity Function

2.3.12 Main Code Portion

```
350 int main()
351 {
352
353     while (1)
354     {
355         system("cls");
356         printf("\033[1;36m===== Sorting Algorithm Visualizer =====\033[0m\n");
357         printf("\n1. Start Visualization");
358         printf("\n2. Exit");
359         printf("\nChoose: ");
360         int choice1;
361         scanf("%d", &choice1);
362         if (choice1 == 2)
363         {
364             printf("\nExiting...\n");
365             break;
366         }
367         int choice;
368         int inputChoice;
369
370         printf("\n1. Manual Input");
371         printf("\n2. Random Array");
372
373         printf("\nChoose: ");
374         scanf("%d", &inputChoice);
375
376         if (inputChoice == 1)
377         |     inputArray();
378         else
379         |     randomArray();
380
381         printf("\nOriginal ");
382         printArray();
383
384         printf("\nChoose Sorting Algorithm\n");
385         printf("1. Bubble Sort\n");
386         printf("2. Selection Sort\n");
387         printf("3. Insertion Sort\n");
388         printf("4. Merge Sort\n");
389         printf("5. Quick Sort\n");
390
391         printf("\nChoose: ");
392         scanf("%d", &choice);
393
394         resetStats();
395     }
```

```

395     clock_t start = clock();
396     switch (choice)
397     {
398     case 1:
399         bubbleSort();
400         break;
401
402     case 2:
403         selectionSort();
404         break;
405
406     case 3:
407         insertionSort();
408         break;
409
410     case 4:
411         mergeSort(0, n - 1);
412         break;
413
414     case 5:
415         quickSort(0, n - 1);
416         break;
417
418     default:
419         printf("Invalid Choice!");
420         exit(0);
421     }
422     clock_t end = clock();
423
424     double timeTaken = (double)(end - start) / CLOCKS_PER_SEC;
425
426     printf("\n\033[1;32mSorting Completed!\033[0m\n");
427
428     printArray();
429
430     printf("\nComparisons : %lld", comparisons);
431     printf("\nSwaps      : %lld", swaps);
432     printf("\nExecution Time : %.6f seconds\n", timeTaken);
433
434     showComplexity(choice);
435
436     printf("\nPress Enter to return to the main menu...");
437     getchar();
438     getchar();
439 }
440 return 0;
441 }

```

Figure 2.12: Main Function

Chapter 3

Performance Evaluation

3.1 Simulation Environment/Simulation Procedure

The Sorting Algorithm Visualizer was tested on a personal computer/laptop. The system had the following specifications:

- Processor: Intel Core i7 (11th Gen)
- Memory (RAM): 16 GB
- Storage: 512 GB SSD
- Operating System: Windows 11 pro (64-bit)
- Compiler: GCC (MinGW) (through Code::VS Code)

The testing procedure was simple. First, the program was compiled using the GCC (MinGW) compiler to check if there were any errors. After that, the program was run and the main menu appeared. Each option from the menu was tested one by one

3.2 Results Analysis/Testing

The program was tested with different array sizes, input types, and sorting algorithms. During testing, the following points were verified:

- Array elements were stored correctly.
- The visualization displayed the array correctly.
- Comparisons and swaps were counted correctly.
- Execution time was measured accurately.
- Each algorithm sorted the array correctly.
- The complexity information was displayed correctly.

Performance Results:

Table 3.1: Algorithm Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Observations:

1. **Bubble Sort:** Easy to understand, shows adjacent comparisons clearly. Good for small arrays.
2. **Selection Sort:** Shows how the minimum element is found and placed. Consistent performance regardless of input.
3. **Insertion Sort:** Works well when the array is nearly sorted. Shows elements being inserted in the correct position.
4. **Merge Sort:** More complex visualization but shows divide-and-conquer approach. Consistent performance.
5. **Quick Sort:** Fast on random data. Performance depends on pivot selection.

3.3 Project Simulation

3.3.1 Manual Input Main Menu

```
===== Sorting Algorithm Visualizer =====  
  
1. Start Visualization  
2. Exit  
Choose: 1  
  
1. Manual Input  
2. Random Array  
Choose: 1  
  
Enter number of elements (1-50): 8  
  
Enter Elements:  
10 5 8 4 3 16 20 25  
  
Original  
Array: 10 5 8 4 3 16 20 25  
  
Choose Sorting Algorithm  
1. Bubble Sort  
2. Selection Sort  
3. Insertion Sort  
4. Merge Sort  
5. Quick Sort  
  
Choice: 1
```

Figure 3.1: Manual Input Main Menu Option

3.3.2 Bubble Sort

```
===== Sorting Algorithm Visualizer =====  
  
3 | ██████████  
4 | ██████████  
5 | ██████████  
8 | ██████████  
10 | ██████████  
16 | ██████████  
20 | ██████████  
25 | ██████████  
  
Comparisons : 19  
Swaps : 9  
  
Sorting Completed!  
  
Array: 3 4 5 8 10 16 20 25  
  
Comparisons : 25  
Swaps : 9  
Execution Time : 0.311000 seconds  
  
===== Bubble Sort =====  
Best Case :  $O(n)$   
Average Case :  $O(n^2)$   
Worst Case :  $O(n^2)$   
Space :  $O(1)$   
=====
```

Figure 3.2: Bubble Sort using Manual Input

3.3.3 Selection Sort

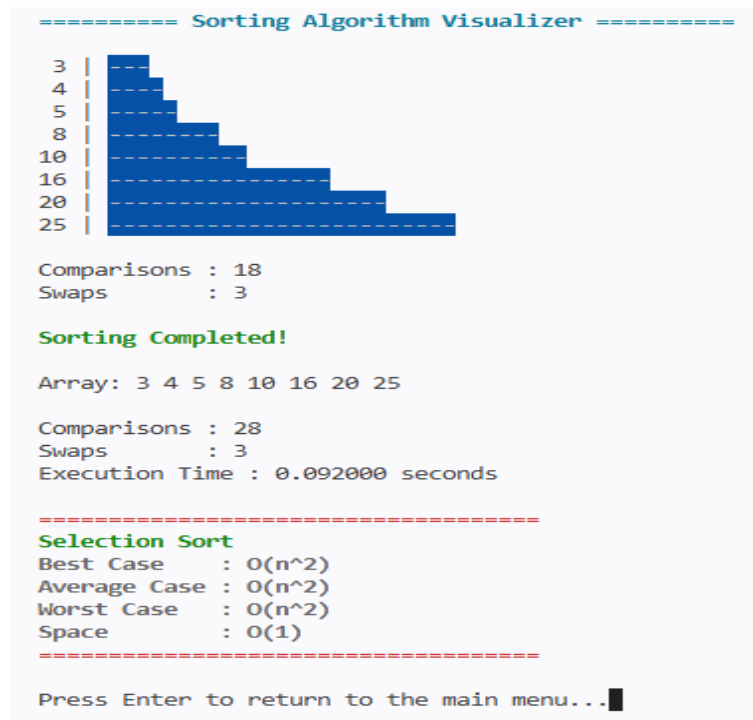


Figure 3.3: Selection Sort using Manual Input

3.3.4 Insertion Sort

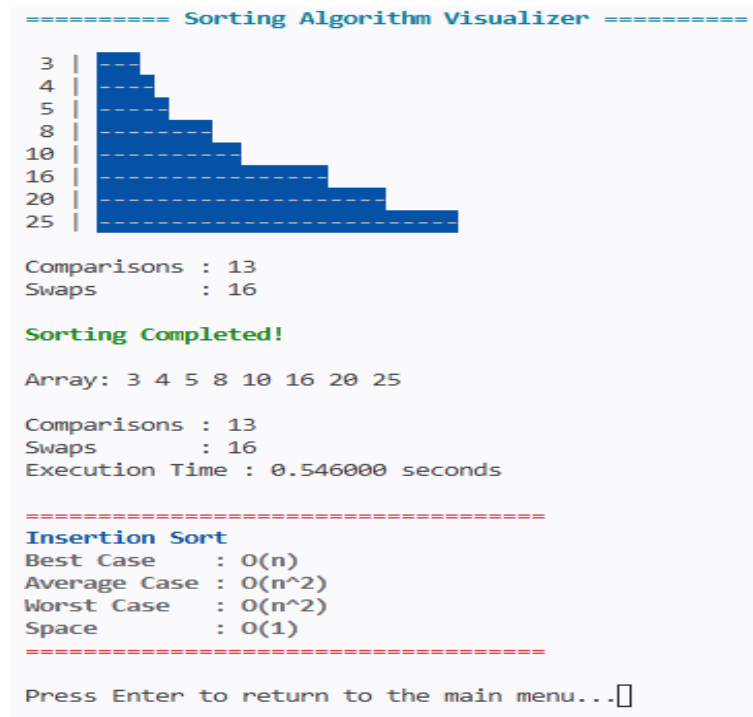


Figure 3.4: Selection Sort using Manual Input

3.3.5 Merge Sort

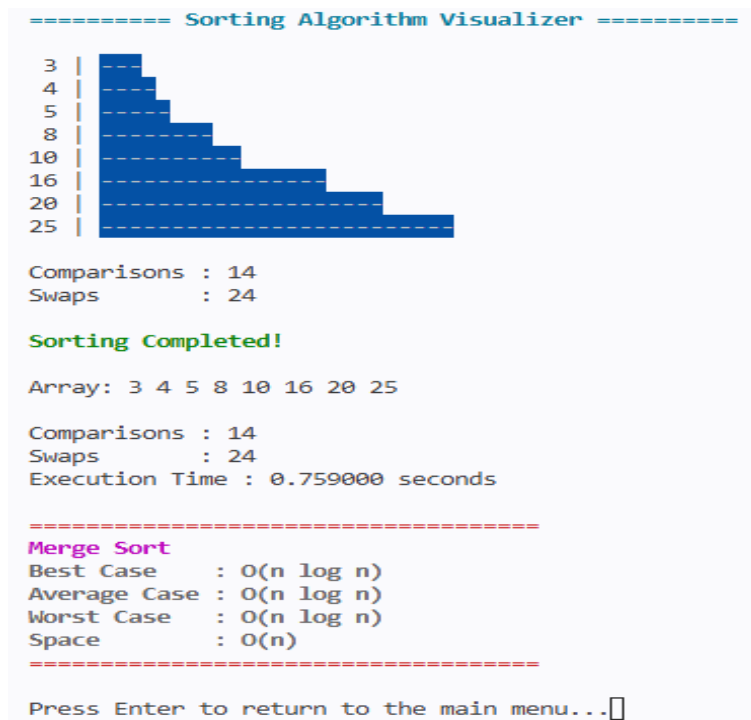


Figure 3.5: Merge Sort using Manual Input

3.3.6 Quick Sort

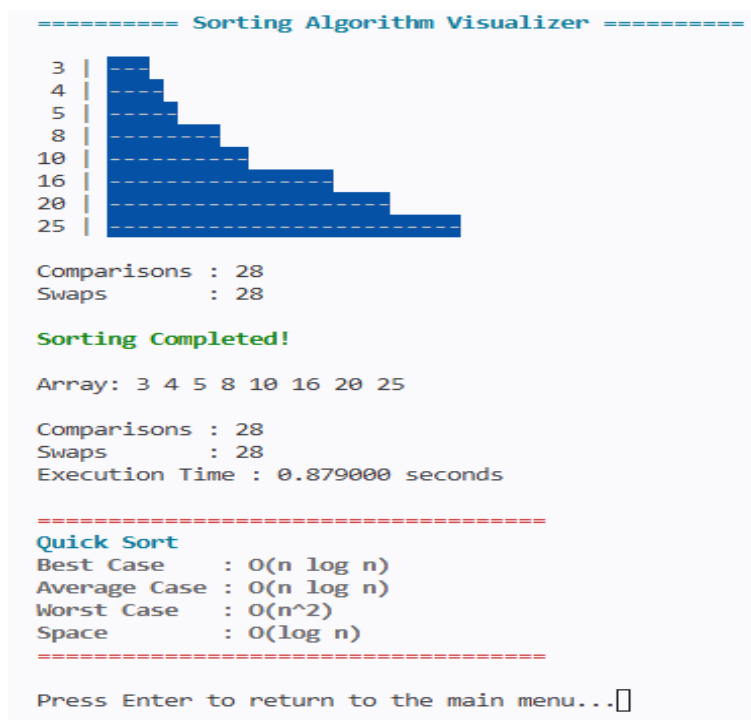


Figure 3.6: Quick Sort using Manual Input

3.3.7 Random Array Main Menu

```
===== Sorting Algorithm Visualizer =====

1. Start Visualization
2. Exit
Choose: 1

1. Manual Input
2. Random Array
Choose: 2

Enter number of elements (1-50): 8

Random Array Generated.

Original
Array: 5 2 4 6 6 11 10 2

Choose Sorting Algorithm
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort

Choice: 1
```

Figure 3.7: Random Array Main Menu Option

3.3.8 End Program Portion

```
===== Sorting Algorithm Visualizer =====

1. Start Visualization
2. Exit
Choose: 2

Exiting...
```

Figure 3.8: End Program

3.4 Results Overall Discussion

The Sorting Algorithm Visualizer performed successfully during testing and produced the expected results. Different array sizes and sorting algorithms were tested. In each case, the program displayed the sorting process correctly.

The program accurately updates the comparisons and swaps for each algorithm. The execution time measurement was also accurate. The visualization helped users understand how each algorithm works step by step.

- **Educational Value:** The visual approach makes it much easier to understand sorting algorithms compared to reading code alone.
- **Algorithm Comparison:** Users can clearly see how different algorithms approach the same sorting problem.
- **Performance Understanding:** The metrics help users understand why some algorithms are faster than others.
- **User Experience:** The console interface is simple and easy to use.
- **Learning Effectiveness:** Step-by-step visualization helps build intuitive understanding of algorithm behavior.

3.4.1 Complex Engineering Problem Discussion

This project involved several engineering challenges during the design and implementation of the Sorting Algorithm Visualizer. The major challenges and their solutions are summarized below:

- **Implementing Multiple Algorithms:** The program successfully implements five different sorting algorithms, each with its own logic and behavior.
- **Real-time Visualization:** The program updates the display after each comparison and swap, making the sorting process visible and easy to follow.
- **Performance Tracking:** The program accurately tracks comparisons and swaps for each algorithm, providing valuable performance insights.
- **User-friendly Interface:** The simple menu-driven interface makes it easy for users to navigate and use the program.
- **Educational Purpose:** The project effectively serves as a learning tool for understanding sorting algorithms.
- **Complexity Display:** The program shows theoretical complexity, helping users connect practical performance with theoretical knowledge.

Chapter 4

Conclusion

4.1 Discussion

The Sorting Algorithm Visualizer project was successfully designed and implemented using the C programming language. The simulator demonstrates the basic operations of sorting algorithms in a simple and interactive way. It allows users to input arrays, choose sorting algorithms, and watch the sorting process step by step.

The project successfully implements five sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. It also displays comparisons, swaps, and execution time after every sorting operation. These features help users understand how sorting algorithms work and how different algorithms compare in terms of efficiency.

Overall, the project achieved its objectives and provides an effective learning tool for studying algorithms.

4.2 Limitations

Although the project works correctly, it has some limitations:

- The maximum array size is limited to 50 elements
- The program runs only in the command-line interface and does not have a graphical user interface (GUI)
- The program only works on Windows due to the use of Windows-specific functions
- Only five sorting algorithms are included
- The visualization speed cannot be controlled by the user
- The program cannot save results or export data
- The program does not include advanced sorting algorithms like Heap Sort or Radix Sort

4.3 Scope of Future Work

The project can be improved by adding more advanced features in the future. Possible future improvements include:

- Graphical User Interface: Develop a GUI for better visualization with colors and animations
- More Sorting Algorithms: Add Heap Sort, Shell Sort, Radix Sort, and Counting Sort
- Speed Control: Allow users to control the speed of visualization
- Cross-Platform Support: Make the program work on Linux and macOS
- Data Persistence: Save and load arrays for future use
- Step-by-Step Mode: Allow users to step through the algorithm manually
- Export Results: Save results and statistics to files
- Algorithm Comparison: Show two algorithms running side by side
- Interactive Learning: Add explanations and pseudocode alongside visualization
- Web Version: Create a web-based version using HTML, CSS, and JavaScript

4.4 Conclusion

The Sorting Algorithm Visualizer project successfully demonstrates the basic concepts of sorting algorithms using the C programming language. It provides a simple way to understand how different sorting algorithms work by showing each step of the sorting process.

The project accurately tracks comparisons, swaps, and execution time while displaying the array state after every operation. This project enhances practical knowledge of algorithms and helps students understand theoretical concepts through hands-on simulation.

The Sorting Algorithm Visualizer serves as a useful educational tool and provides a strong foundation for developing more advanced sorting algorithm visualizers in the future.

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [2] Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- [3] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- [4] Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C* (4th ed.). Pearson.
- [5] C Programming Language Documentation. (2026). *GNU C Library Reference Manual*.